

AI-Enhanced Fast Cosmological Simulations for Accurate Large-Scale Structure Inference

David Amrein, Jozef Bucko, Alexandre Refregier

July 8, 2026

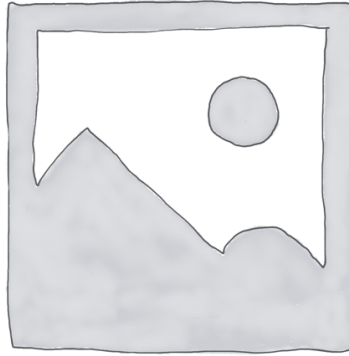


Figure 1: cover picture

Contents

1	Introduction	3
2	Theoretical Background	3
2.1	Cosmological Framework	3
2.1.1	The Cosmic Web	3
2.1.2	Matter Power Spectrum and Transfer Function	3
2.1.3	Lightcones and Snapshots	5
2.2	Cosmological Simulations	5
2.2.1	DISCO-DJ	5
2.2.2	Pkdgrav	6
2.2.3	CosmoGridV1	7
3	Methods and implementation	7
3.1	Pipeline Architecture	7
3.2	IC generation	8
3.2.1	Validation against CosmoGridV1	8
3.2.2	Backscaled initial conditions and hand-off to DISCO	8
3.3	Shell Builder	8
3.3.1	Core	8
3.3.2	Crossing Condition	9
3.4	Neural Network	10
3.4.1	Dimensional Reduction	11
3.4.2	Data preparation	12
3.4.3	Network Architecture	12
3.4.4	Training Procedure	12
3.4.5	Validation Strategy	12
4	Computational Infrastructure	12
4.1	Computing Environments	12
4.1.1	Local Development	12
4.1.2	Euler (ETH Zurich)	13
4.1.3	CSCS (Alps / Piz Daint)	13
4.2	Software Engineering	13
4.2.1	Code Structure	13
4.2.2	Job Scheduling and Submission	14
4.2.3	Job Scheduling and Submission	14
4.2.4	Version Control (GitHub)	15
5	Results	15
6	Conclusion	15
A	Implementation Details	17
A.1	Job Configuration	17

1 Introduction

2 Theoretical Background

2.1 Cosmological Framework

2.1.1 The Cosmic Web

On scales of a few to a few hundred megaparsecs, matter in the Universe is not smoothly distributed but arranged into an interconnected network of dense compact knots, elongated filaments, two-dimensional sheets, and large under-dense voids – collectively known as the *cosmic web* [2, 13]. The term originates with Bond, Kofman, and Pogosyan, who showed that this filament-dominated pattern is already present in embryonic form in the tidal field of the initial density perturbations, with the nonlinear gravitational evolution sharpening rather than creating it [2]. The same gravitational instability that seeds this pattern is responsible for its growth: initially small density fluctuations are amplified by their own self-gravity, so that overdense regions contract while voids expand and come to dominate the volume of the Universe [2]. Large galaxy redshift surveys – 2dFGRS, SDSS, and the 2MASS redshift survey (2MRS) among them – as well as cosmological N -body simulations both recover this same web-like arrangement of galaxies and dark matter [13].

Gravitational instability drives the emergence of anisotropic structure: because collapse along the direction of a region’s strongest tidal compression proceeds fastest, an initially only mildly non-spherical overdensity first flattens into a sheet, then contracts further into a filament, and only finally collapses fully into a compact halo – the sequence of anisotropic collapse first described by Zel’dovich [13]. This picture can be made quantitative by looking at the local tidal (or velocity-shear) field: following the tidal-tensor classification reviewed by [13], one computes the Hessian of the (rescaled) gravitational potential ϕ ,

$$T_{\alpha\beta} = \frac{\partial^2 \phi}{\partial x_\alpha \partial x_\beta}, \quad \nabla^2 \phi = \delta, \quad (1)$$

where $\delta = (\rho - \bar{\rho})/\bar{\rho}$ is the dimensionless matter overdensity and ϕ has been normalised by $4\pi G\bar{\rho}$ so that Eq. (1) takes the form of the Poisson equation [13]. Diagonalising the (real, symmetric) tidal tensor at each point gives three eigenvalues $\lambda_1 > \lambda_2 > \lambda_3$; counting how many of them exceed a chosen threshold λ_{th} classifies that point as part of a void (zero eigenvalues above threshold), a sheet (one), a filament (two), or a knot (three) [13]. Voids, sheets, filaments and knots are thus not four independent morphologies but a nested hierarchy, each embedded within the next.

2.1.2 Matter Power Spectrum and Transfer Function

While the tidal-tensor classification above describes the cosmic web *qualitatively*, its statistical properties are quantified through the matter power spectrum $P(k)$. Writing the density contrast in Fourier space as $\delta(\mathbf{k})$ and assuming statistical homogeneity and isotropy, the power spectrum is defined through

$$\langle \delta(\mathbf{k}) \delta^*(\mathbf{k}') \rangle = (2\pi)^3 \delta_D(\mathbf{k} - \mathbf{k}') P(k), \quad (2)$$

where δ_D is the Dirac delta and the angle brackets denote an ensemble average over realisations of the initial conditions [4, 9]. In a simplified form it can be written as

$$P(k) \propto k^{n_s} T^2(k), \quad (3)$$

with n_s the scalar spectral index [5, 9]. The transfer function $T(k)$ can in turn be approximated by

$$T(k) = 1 - \frac{k}{k_{\text{eq}}} + \dots, \quad k_{\text{eq}} = H_0 \sqrt{\frac{2\Omega_m}{\Omega_r}}, \quad (4)$$

[1, 5]. The transfer function is therefore an essential ingredient of the matter power spectrum, alongside the matter and radiation densities, and it also depends on the neutrino mass, as illustrated in Figure 2.

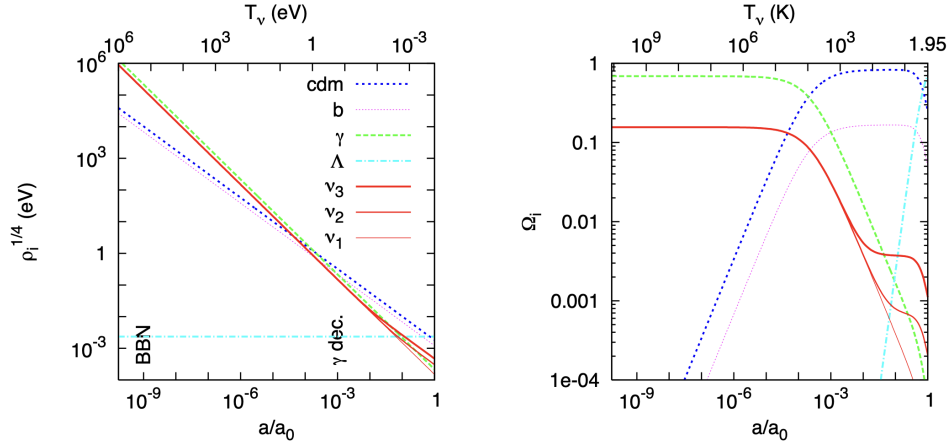


Figure 2: Linear matter transfer function for three neutrino masses (red curves). Reproduced from [11].

The three red curves correspond to the neutrino masses $m_{\nu_1} = 0$, $m_{\nu_2} = 0.009$ and $m_{\nu_3} = 0.05$ eV [12, 11]. This in turn affects $P(k)$, as also the angular power spectrum C_ℓ , which is defined as

$$C_\ell \approx \int k^2 P(k) dk, \quad (5)$$

and is essentially the Fourier transform of the $P(k)$ [16], illustrated for varying neutrino mass in Figure 3.

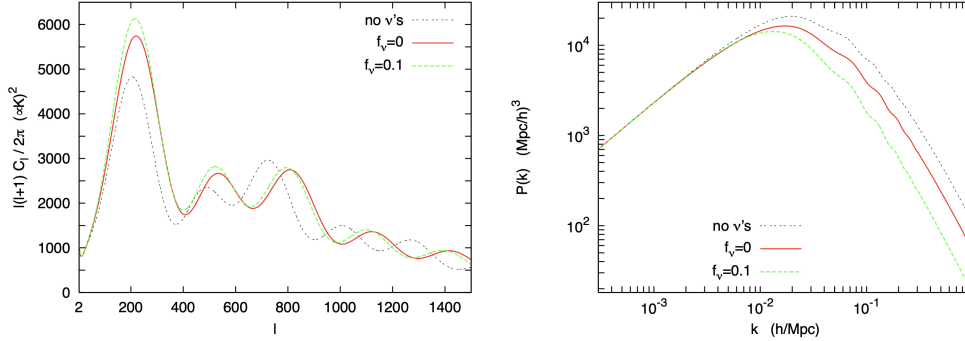


Figure 3: Effect of the neutrino mass on the angular power spectrum. Reproduced from [11].

The dependence on the neutrino mass is not a side note here: it reappears directly when generating the initial conditions for the simulations in Sec. 3.2, since matching the transfer function – neutrinos included – is

a precondition for those initial conditions to be consistent with CosmoGridV1. More generally, obtaining $P(k)$ precisely, including this transfer function, requires solving the coupled linear Einstein–Boltzmann equations for the perturbed Friedmann–Lemaître–Robertson–Walker metric and the phase-space distributions of all matter and radiation species – the same system solved by the differentiable Einstein–Boltzmann module of the Disco-DJ framework used later in this thesis [8].

2.1.3 Lightcones and Snapshots

An N -body simulation naturally outputs *snapshots*: the full particle phase-space (positions and velocities) within a periodic box of fixed comoving side length, at a fixed time. Reproducing how we actually observe the Universe, however, requires a *lightcone*: rather than a single time slice, an observer at a fixed spacetime point receives light emitted from progressively more distant (and hence earlier) shells as one looks to higher redshift, so a simulated lightcone must record particles at the (redshift-dependent) comoving distance from which they are being observed, not at a single global time.

Because the depth of the desired lightcone (out to some z_{max}) is typically larger than a single simulation box, the box is periodically replicated around a centrally placed observer: box replication takes advantage of the fact that a simulation with periodic boundary conditions can, in principle, represent an arbitrarily large volume, at the cost of the same structures reappearing at multiple replica images along a given line of sight [10]. This replication strategy, and the trade-off between box size, structure repetition, and the resulting bias or excess variance in the projected lightcone statistics, is discussed in detail by [10], who additionally introduce a shell-permutation scheme that mitigates the repeated-structure problem by drawing different radial shells of a single lightcone from independent simulations of the same cosmology. The shell builder introduced in Sec. 3.3.1 implements the same replication idea directly inside the time integration, rather than as a post-processing step.

2.2 Cosmological Simulations

Cosmological N -body simulations have historically tracked, and in some respects driven, the development of high-performance computing: the number of particles that can be evolved in a fixed wall-clock time has grown by many orders of magnitude over the past decades, both because of raw hardware improvements and because of continual gains in the efficiency of the gravity solvers themselves [15]. The scientific payoff of these simulations lies in their ability to model the large-scale structure (LSS) of the Universe to the non-linear precision that photometric surveys require. LSS observables are particularly sensitive to the present-day matter density Ω_m and the amplitude of matter clustering σ_8 ; because a lightcone simulation can be sliced into tomographic redshift bins, it further allows the growth of structure to be tracked over cosmic time, which is what makes such simulations sensitive to the dark-energy equation-of-state parameter w_0 (and its possible time evolution) and hence to the physical origin of cosmic acceleration [10]. Two solvers embody opposite ends of the resulting accuracy-versus-cost trade-off, and both are used in this thesis: the tree-based PKDGRAV3 (Sec. 2.2.2), which anchors the accurate CosmoGridV1 reference simulations, and the particle-mesh Disco-DJ (Sec. 2.2.1), which is fast enough to generate the large volume of training data our approach requires.

2.2.1 DISCO-DJ

Disco-DJ (*Differentiable Simulations for Cosmology – Done with JAX*) is a particle-mesh (PM) code whose entire forward model is GPU-accelerated and differentiable, so that gradients of any output with respect to the input parameters – or the initial conditions themselves – are available through automatic differentiation [14]. Like any N -body method it approximates the collisionless dynamics of dark matter, i.e. a solution of

the cosmological Vlasov–Poisson system for the phase-space distribution $f(\mathbf{x}, \mathbf{p}, t)$,

$$\frac{\partial f}{\partial t} + \frac{\mathbf{p}}{a^2} \cdot \nabla_{\mathbf{x}} f - \nabla_{\mathbf{x}} \phi \cdot \nabla_{\mathbf{p}} f = 0, \quad \nabla_{\mathbf{x}}^2 \phi = \frac{3 \Omega_m H_0^2}{2a} \delta, \quad (6)$$

where $\mathbf{x}$ are comoving coordinates, $\mathbf{p} = a^2 \dot{\mathbf{x}}$ the conjugate momentum, $a(t)$ the scale factor, and ϕ the peculiar gravitational potential sourced by the density contrast δ through the Poisson equation [14].

What sets the PM scheme apart from the tree/FMM solver of Sec. 2.2.2 is how the potential is obtained: rather than summing particle-by-particle, the particles are deposited onto a regular mesh to give δ , the Poisson equation is solved in Fourier space with a single FFT (where $\nabla^2 \rightarrow -k^2$), and the force $-\nabla_{\mathbf{x}} \phi$ is interpolated back to each particle [14]. A force evaluation is thus global and cheap, at the price of a spatial resolution limited by the mesh spacing – the origin of the small-scale inaccuracy discussed in Sec. 3.1, and, as shown in Sec. 4.1.1, also the origin of Disco-DJ’s memory footprint on a cluster.

2.2.2 Pkdgrav

The gravity solver PKDGRAV3 takes the opposite approach: rather than a mesh, it computes the gravitational force with a binary-tree Fast Multipole Method (FMM). Rather than only approximating a distant tree cell’s effect on individual particles (as in a Barnes–Hut tree code), it also expands the potential *within* a receiving cell sourced by a distant cell, an interaction that lets the cost of a full force evaluation scale as $\mathcal{O}(N)$ instead of $\mathcal{O}(N \log N)$ [15].

This accuracy is not free: because particles in different regions of a cosmological box span many orders of magnitude in local density, and hence in dynamical time, a single global time step would be highly wasteful. PKDGRAV3 instead assigns each particle to one of several time-step “rungs”, advancing only the particles active on a given rung at each sub-step, and additionally builds a second, smaller tree restricted to the small subset of particles on the shortest (“very active”) rungs so that this frequently-rebuilt tree stays cheap; the combination is summarised by the kick-drift-kick “umbrella” diagram over one base time step, shown in Figure 4. This dual-tree multi-stepping scheme is what keeps the tree-construction cost from dominating the runtime even though the shortest rungs may be walked hundreds of times per base step [15].

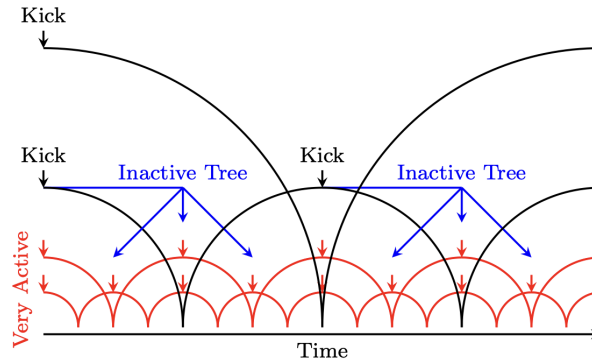


Figure 4: [15]

It is precisely this combination of a per-particle tree, per-particle rung state, and domain-decomposition buffers that gives PKDGRAV3 the larger, and rather different, memory footprint discussed quantitatively in Sec. 4.1.1.

2.2.3 CosmoGridV1

CosmoGridV1 is a large, publicly released suite of lightcone N -body simulations built specifically for map-level (rather than two-point-function-only) cosmological inference from large-scale-structure probes such as weak lensing and galaxy clustering [10]. It samples a six-dimensional flat w CDM parameter space (Ω_m , σ_8 , w_0 , H_0 , n_s , Ω_b , with a fixed total neutrino mass) at 2500 points arranged on a Sobol sequence, running 7 independent realisations at every point, in addition to 200 realisations (with parameter-derivative pairs) at a fixed fiducial cosmology and a smaller set of benchmark runs at higher resolution used to validate the main configuration’s choices of box size, particle count, and shell thickness [10]. Every simulation is run with PKDGRAV3 in a periodic box of 900 Mpc/ h with 832^3 particles, and stores its lightcone as HEALPix particle-count maps at $N_{\text{side}} = 2048$ out to $z = 3.5$, together with friends-of-friends halo catalogues at each output step [10]. These raw particle-count shells are the basis from which observable maps of weak-lensing convergence, galaxy clustering, and intrinsic alignment are subsequently constructed, and the same dataset has previously been used, together with a shell-based baryon correction model, for a deep-learning cosmological parameter analysis of the KiDS-1000 weak-lensing survey [7]. It is this suite – box size, particle count, cosmology, and all – that Sec. 3.1 takes as the accuracy target for the pipeline developed in this thesis.

3 Methods and implementation

3.1 Pipeline Architecture

The reference suite CosmoGridV1 evolves every realisation with PKDGRAV3 (Sec. 2.2.2), whose FMM force solver and adaptive time-stepping resolve structure accurately down to small scales. That accuracy is expensive: each lightcone costs many node-hours, so the millions of realisations required for map-level, simulation-based inference dominate the total cost of the analysis [10, 15].

Disco-DJ offers the opposite trade-off. Its particle-mesh solver (Sec. 2.2.1) reproduces the same large-scale clustering in minutes rather than hours, but the force resolution is capped by the PM grid, so power is progressively lost on small scales – exactly the scales that carry much of the non-Gaussian information exploited by the deep-learning analysis [14].

The aim of this work is therefore to keep Disco-DJ’s speed while recovering the small-scale content that PKDGRAV3 would have produced. Formally, we seek a correction f acting on the cheap lightcone maps such that

$$f(\text{Map}_{\text{DISCO}}) \approx \text{Map}_{\text{COSMOGRID}}, \quad (7)$$

where both maps are generated from the *same* initial conditions and cosmology, so that f has only to model the resolution-dependent difference between the two solvers, not any change of realisation.

Approaches. Two routes to f were considered. The first is a direct, image-level neural-network correction that maps $\text{Map}_{\text{DISCO}}$ onto $\text{Map}_{\text{COSMOGRID}}$ pixel by pixel. This is expressive, but heavy: full-sky HEALPix maps at $N_{\text{side}} = 2048$ ($\sim 5 \times 10^7$ pixels) are large, live on the sphere rather than on a flat grid, and do not fit on a single GPU without tiling into patches (Sec. 3.4). The second route, adopted here, works in harmonic space and corrects the angular power spectrum C_ℓ of the Disco-DJ maps at high multipole ℓ , where the PM resolution deficit is concentrated. Operating on C_ℓ is far cheaper, is naturally defined on the sphere, and targets the deficit exactly where Eq. (5) relates it to $P(k)$; the trade-off is that it corrects a summary statistic rather than the full field.

3.2 IC generation

3.2.1 Validation against CosmoGridV1

Before using DISCO’s shell builder (Sec. 3.3.1) for any downstream analysis, we first validate that a lightcone produced with our PKDGRAV3 configuration reproduces the reference CosmoGridV1 statistics. To this end we run PKDGRAV3 with its native lightcone module on the exact box, resolution, and cosmology used by CosmoGridV1, and compare the resulting matter angular power spectrum C_ℓ against the corresponding CosmoGridV1 lightcone.

3.2.2 Backscaled initial conditions and hand-off to disco

As described in Sec. 2.1.2, the initial conditions must be built from a correct transfer function evaluated at the correct neutrino mass, since this directly determines whether the resulting field is consistent with CosmoGridV1. The resulting particle positions and velocities at $a_i = 1/(1 + z_i)$ are exported and used, unmodified, as the initial condition for the corresponding DISCO run: DISCO evolves this state forward to $a = 1$ and accumulates the lightcone with the shell builder of Sec. 3.3.1 over the same redshift range, $z_{\max} = 3.5$ down to $z = 0$, at the same HEALPix resolution $N_{\text{side}} = 2048$.

3.3 Shell Builder

3.3.1 Core

A lightcone output requires the particle distribution to be resampled at every integration step, since the observable past lightcone shell changes continuously as the universe evolves. The most direct implementation stores the full particle snapshot at each of the n_{steps} integration steps and performs the shell decomposition after the simulation has finished. This approach is memory-prohibitive: retaining n_{steps} full-resolution snapshots simultaneously exceeds the available device memory for the particle counts considered here. We therefore interleave the shell decomposition with the time integration, so that at any given time only the particle states immediately before and after the current integration step, together with a set of low-dimensional pixelized count maps, need to be held in memory.

Time discretization. The radial shells are not defined by the integrator itself but are supplied externally, e.g. through a CosmoGridV1-format meta-info table. Each shell $s \in \{1, \dots, S\}$ is specified by a redshift interval $[z_s^{\text{lo}}, z_s^{\text{hi}}]$ and the corresponding comoving-distance interval $[r_s^{\text{lo}}, r_s^{\text{hi}}]$. To ensure that no shell boundary is crossed *within* an integration step, we define the set of integration times as the union of all shell edges,

$$\{z_j\}_{j=0}^S = \text{unique}\left(\{z_s^{\text{lo}}\}_{s=1}^S \cup \{z_s^{\text{hi}}\}_{s=1}^S\right), \quad a_j = \frac{1}{1 + z_j}, \quad (8)$$

sorted in ascending order of a_j (equivalently, descending order of z_j). By construction, every shell boundary coincides exactly with an integration-step boundary. The interval $[a_{\text{ini}}, a_0]$, preceding the first shell, is integrated separately with n_{pre} sub-steps and without shell accumulation, in order to bring the particle state into registry with the shell grid defined in Eq. (8). The remaining $J = S$ intervals $[a_j, a_{j+1}]$, $j = 0, \dots, J - 1$, are each advanced with exactly one integration sub-step.

Accumulation procedure. Algorithm 1 summarizes the per-step accumulation. At step j , the comoving positions and validity mask of all particles are recorded both before the sub-step, $\{\mathbf{x}_p^{(j)}, m_p^{(j)}\}$, and after it, $\{\mathbf{x}_p^{(j+1)}, m_p^{(j+1)}\}$. The scale-factor edges of the step are converted to comoving distance through the cosmology-dependent map $\chi(a)$,

$$R_j = \chi(a_j), \quad R_{j+1} = \chi(a_{j+1}), \quad R_j > R_{j+1}, \quad (9)$$

where the inequality follows from χ being strictly decreasing on $a \in (0, 1]$. The set of shells that are traversed during step j is

$$\mathcal{A}_j = \{s : r_s^{\text{lo}} < R_j \wedge r_s^{\text{hi}} > R_{j+1}\}, \quad (10)$$

i.e. those shells whose radial bin overlaps the annulus $[R_{j+1}, R_j)$ swept out during the step. For each active shell $s \in \mathcal{A}_j$, the pre- and post-step particle states, together with the radial bounds $(r_s^{\text{lo}}, r_s^{\text{hi}})$, are passed to the shell-accumulation routine described in Sec. 3.3.2, which returns a HEALPix pixel count map $\mathbf{M}_s^{(j)} \in \mathbb{N}^{N_{\text{pix}}}$. This map is added to a running total,

$$\mathbf{S}_s += \mathbf{M}_s^{(j)}, \quad s \in \mathcal{A}_j. \quad (11)$$

Because the integration grid is constructed from the shell edges themselves (Eq. 8), $\mathcal{A}_j$ reduces to a single shell in the generic case; the overlap criterion in Eq. (10) is retained in its general form to also cover degenerate or adjacent bins in the input shell table.

Algorithm 1 Streaming shell accumulation

```

1: Load shell table  $\{(z_s^{\text{lo}}, z_s^{\text{hi}}, r_s^{\text{lo}}, r_s^{\text{hi}})\}_{s=1}^S$ ; build integration grid  $\{a_j\}_{j=0}^J$  (Eq. 8)
2: Initialize  $\mathbf{S}_s \leftarrow \mathbf{0}$  for all  $s$ 
3: Integrate particle state from  $a_{\text{ini}}$  to  $a_0$  with  $n_{\text{pre}}$  sub-steps (no accumulation)
4: for  $j = 0, \dots, J - 1$  do
5:   Record pre-step positions and mask,  $\{\mathbf{x}_p^{(j)}, m_p^{(j)}\}$ 
6:   Integrate one sub-step:  $a_j \rightarrow a_{j+1}$ 
7:   Record post-step positions and mask,  $\{\mathbf{x}_p^{(j+1)}, m_p^{(j+1)}\}$ 
8:   Compute  $R_j, R_{j+1}$  (Eq. 9) and active shells  $\mathcal{A}_j$  (Eq. 10)
9:   for  $s \in \mathcal{A}_j$  do
10:     $\mathbf{M}_s^{(j)} \leftarrow$  crossing map from  $\{\mathbf{x}_p^{(j)}, \mathbf{x}_p^{(j+1)}, m_p^{(j)}, m_p^{(j+1)}\}$  restricted to  $[r_s^{\text{lo}}, r_s^{\text{hi}})$  (Sec. 3.3.2)
11:     $\mathbf{S}_s += \mathbf{M}_s^{(j)}$ 
12:   end for
13:   Discard pre-step state; retain post-step state as pre-step state for  $j + 1$ 
14: end for
15: return  $\{\mathbf{S}_s\}_{s=1}^S$ 

```

Output. Once all J steps have been processed, the set of per-shell count maps $\{\mathbf{S}_s\}_{s=1}^S$, together with the shell table, is serialized to a single output file. At no point during the procedure are more than $\mathcal{O}(1)$ full particle snapshots and S integer-valued maps of size N_{pix} held simultaneously; this is what removes the memory bottleneck associated with the naive end-of-run decomposition.

3.3.2 Crossing Condition

The condition follows the moving-lightcone formulation of PKDGRAV3 [6] exactly, rather than the vector-trajectory intersection used in the earlier sketch.

Periodic replicas. The observer is placed at the box centre, $\mathbf{o} = (L/2, L/2, L/2)$, with L the box side length. Since the requested lightcone depth $\chi_{\text{max}} = \chi(1/(1 + z_{\text{max}}))$ generally exceeds L , the simulation volume is periodically replicated: let

$$\mathcal{D} = \left\{ \mathbf{d} = (n_1, n_2, n_3) L : \mathbf{n} \in \mathbb{Z}^3, \text{ replicated cube intersects the ball of radius } \chi_{\text{max}} \text{ about } \mathbf{o} \right\} \quad (12)$$

denote the finite set of replica offsets, precomputed once from $(\chi_{\max}, L)$. For each $\mathbf{d} \in \mathcal{D}$ we also precompute the nearest and farthest possible distance of the replicated cube from the observer,

$$r_{\min}(\mathbf{d}) = \|\max(0, |\mathbf{d}| - L/2)\|, \quad r_{\max}(\mathbf{d}) = \|\mathbf{d}| + L/2\|, \quad (13)$$

(componentwise max and absolute value, followed by the Euclidean norm), which allow a replica to be rejected for a given step without evaluating any particle if its radial range cannot overlap the step's lightcone annulus.

Linear crossing formula. For step j with N-body boundaries $a_j < a_{j+1}$, define, as before, $r_j^{\text{hi}} = \chi(a_j)$ and $r_j^{\text{lo}} = \chi(a_{j+1})$. For a particle with pre-/post-step positions $\mathbf{x}_p(a_j), \mathbf{x}_p(a_{j+1})$ relative to the box centre, and a replica offset $\mathbf{d} \in \mathcal{D}$, define the shifted positions and their radii

$$\mathbf{R}_0 = \mathbf{x}_p(a_j) + \mathbf{d}, \quad \mathbf{R}_1 = \mathbf{x}_p(a_{j+1}) + \mathbf{d}, \quad r_0 = \|\mathbf{R}_0\|, \quad r_1 = \|\mathbf{R}_1\|. \quad (14)$$

The implementation follows PKDGRAV3 and interpolates the particle radius $r_p(v)$ and the lightcone front radius $R(v)$ linearly over the step,

$$r_p(v) = r_0 + v(r_1 - r_0), \quad R(v) = r_j^{\text{hi}} - v(r_j^{\text{hi}} - r_j^{\text{lo}}), \quad v \in [0, 1], \quad (15)$$

and solves $r_p(v^*) = R(v^*)$ for the crossing parameter,

$$v^* = \frac{r_j^{\text{hi}} - r_0}{(r_j^{\text{hi}} - r_j^{\text{lo}}) + (r_1 - r_0)}. \quad (16)$$

A particle–replica pair is accepted only if $v^* \in [0, 1]$; since Eq. (16) partitions the step interval, this guarantees that every particle is caught in *at most one* step, for a fixed replica – evaluating the crossing test against the (generally narrower) metainfo shell bounds instead of the full step bounds $[r_j^{\text{lo}}, r_j^{\text{hi}}]$ would break this partition and could double-count particles moving across two adjacent shells within the same step. Pairs with $|r_1 - r_0|$ below a numerical tolerance (10^{-20}) are treated as non-moving and rejected, and the denominator of Eq. (16) is floored in magnitude at 10^{-10} to avoid division by a near-zero value.

Shell assignment and pixelization. The crossing position is obtained by applying v^* to the *vector* positions,

$$\mathbf{x}_{\text{cross}} = (1 - v^*)\mathbf{R}_0 + v^*\mathbf{R}_1, \quad r_{\text{cross}} = \|\mathbf{x}_{\text{cross}}\|, \quad (17)$$

and the pair is finally kept only if its crossing radius falls inside the metainfo (or, in the absence of an override, the step) shell bounds,

$$r_s^{\text{lo}} \leq r_{\text{cross}} < r_s^{\text{hi}}. \quad (18)$$

Surviving crossings are pixelized with the equal-area HEALPix ring scheme at resolution N_{side} from the unit vector $\hat{\mathbf{x}}_{\text{cross}} = \mathbf{x}_{\text{cross}}/r_{\text{cross}}$, and the corresponding pixel of the shell count map is incremented by one. Algorithm 2 summarizes the full test, which is applied to every particle and every replica $\mathbf{d} \in \mathcal{D}$ surviving the fast reject of Eq. (13), for every step j and every active shell $s \in \mathcal{A}_j$.

This procedure reproduces the corner-observer, box-replication scheme of PKDGRAV3 [6] almost exactly, with two differences: the observer is placed at the box centre rather than a corner, and the replica set $\mathcal{D}$ (Eq. 12) is sized adaptively from the requested depth $\chi_{\max}$ and box length L – together with a per-step, per-replica radial reject (Eq. 13) – rather than fixed a priori to $6^3 = 216$ images.

3.4 Neural Network

The correction f of Eq. (7) is implemented as a neural network operating on the angular power spectra of the Disco-DJ lightcone maps produced by the pipeline above. The remainder of this section works through that network end to end: how the spherical input is made tractable on a single GPU (Sec. 3.4.1), how training pairs are assembled from matched Disco-DJ/CosmoGridV1 realisations (Sec. 3.4.2), the architecture itself (Sec. 3.4.3), how it is trained (Sec. 3.4.4), and how the corrected maps are validated (Sec. 3.4.5).

Algorithm 2 Lightcone crossing test (per step j , per replica $\mathbf{d}$)

Require: positions $\mathbf{x}_p(a_j), \mathbf{x}_p(a_{j+1})$; step radii $r_j^{\text{lo}}, r_j^{\text{hi}}$; shell bounds $r_s^{\text{lo}}, r_s^{\text{hi}}$; replica $\mathbf{d}$

```

1:  $\mathbf{R}_0 \leftarrow \mathbf{x}_p(a_j) + \mathbf{d}, \quad \mathbf{R}_1 \leftarrow \mathbf{x}_p(a_{j+1}) + \mathbf{d}$ 
2:  $r_0 \leftarrow \|\mathbf{R}_0\|, \quad r_1 \leftarrow \|\mathbf{R}_1\|$ 
3: if  $|r_1 - r_0| \leq \epsilon_{\text{move}}$  then
4:   return reject (stationary)
5: end if
6:  $v^* \leftarrow (r_j^{\text{hi}} - r_0) / [(r_j^{\text{hi}} - r_j^{\text{lo}}) + (r_1 - r_0)]$  (Eq. 16, denominator floored at  $\epsilon_{\text{den}}$ )
7: if  $v^* \notin [0, 1)$  then
8:   return reject (not caught this step)
9: end if
10:  $\mathbf{x}_{\text{cross}} \leftarrow (1 - v^*)\mathbf{R}_0 + v^*\mathbf{R}_1, \quad r_{\text{cross}} \leftarrow \|\mathbf{x}_{\text{cross}}\|$ 
11: if  $r_{\text{cross}} \notin [r_s^{\text{lo}}, r_s^{\text{hi}})$  then
12:   return reject (wrong shell)
13: end if
14:  $\text{pix} \leftarrow \text{VEC2PIX}(N_{\text{side}}, \hat{\mathbf{x}}_{\text{cross}})$ 
15:  $\mathbf{S}_s[\text{pix}] += 1$ 

```

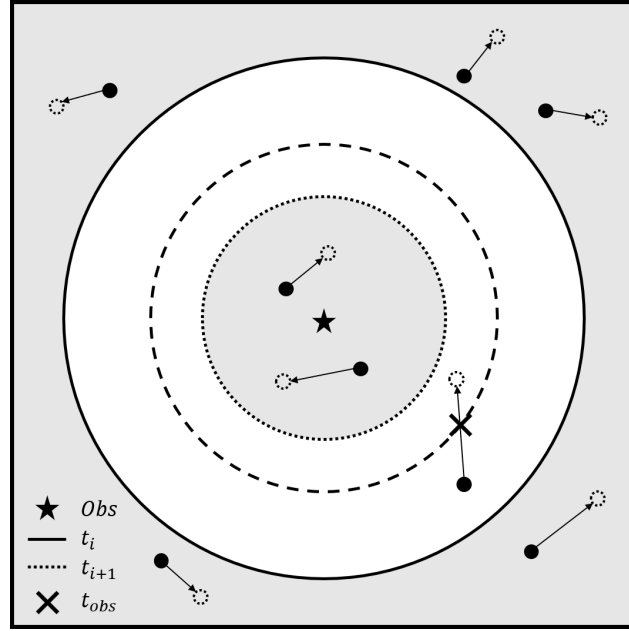


Figure 5: [6]

3.4.1 Dimensional Reduction

As noted in Sec. 3.1, a full-resolution HEALPix map at $N_{\text{side}} = 2048$ does not fit on a single GPU alongside a network of any depth. Two dimensional-reduction strategies address this VRAM constraint: tiling the sphere into flat patches small enough to fit standard convolutional layers, or working directly with a spherical-harmonic (or graph-based spherical-CNN) representation that respects the sphere's topology without an explicit flat projection.

3.4.2 Data preparation

3.4.3 Network Architecture

Spherical convolutional architectures such as DeepSphere [3], which operate directly on the HEALPix pixelization via a graph representation, are a natural starting point for a network that must respect the spherical geometry motivated in Sec. 3.4.1.

3.4.4 Training Procedure

3.4.5 Validation Strategy

The corrected maps are validated against the CosmoGridV1 reference in the same summary statistics the correction targets: the (angular) power spectrum C_ℓ of Eq. (5), and convergence (κ) maps projected with UFALCON, so that the comparison is made on the same observable quantities used to build the training objective in Sec. 3.4.4.

4 Computational Infrastructure

4.1 Computing Environments

Development proceeded through three tiers of increasing scale: a local workstation for prototyping, ETH Zurich’s Euler cluster for medium-scale production, and the CSCS Alps system for full-resolution runs. Each tier trades a shorter debugging loop for less computational headroom, and it is the memory footprint of the two codes from Sec. 2.2.1 and Sec. 2.2.2 – not their runtime – that ultimately decides which tier a given job belongs to.

4.1.1 Local Development

DISCO and PKDGRAV3 were first set up on a local machine to establish a minimal working configuration and to inspect their output interactively: results can be plotted and checked immediately, without the queueing latency of a batch system, which shortens the debugging loop considerably. In practice this means that a bug in, say, the shell-builder logic of Sec. 3.3.1 or in the initial-condition hand-off of Sec. 3.2 is caught and fixed in seconds on a laptop, rather than only surfacing after a queued job on a cluster has already run to completion. Small tests at a resolution of $16^3 = 4096$ particles complete in under a minute on a single core; a run this size is not meant to be physically meaningful, only to exercise the full pipeline – initial conditions, time integration, and lightcone accumulation – end to end before committing to a production-scale configuration.

The binding constraint once the resolution is scaled up to production size is memory, and the two codes hit that limit for structurally different reasons. Storing only the raw phase-space state of a particle – three position and three velocity components, each a single-precision (4-byte) float, i.e. $6 \times 4 = 24$ bytes per particle – for a CosmoGridV1-sized run gives

$$N_p = 832^3 \approx 5.8 \times 10^8 \text{ particles}, \quad N_p \times 24 \text{ bytes} \approx 14 \text{ GB}, \quad (19)$$

which already exceeds a typical workstation’s RAM once any headroom for the rest of the run is included. This number is only a floor, and the two codes exceed it in different ways. For PKDGRAV3, Eq. (19) covers only the particle positions and velocities themselves; as already noted in Sec. 2.2.2, the code additionally carries a per-particle tree, per-particle rung state, and domain-decomposition buffers alongside this raw data, so its actual memory footprint at a given particle count is correspondingly larger than Eq. (19), by an amount that grows with the depth of the tree and the number of particles active on the shortest rungs at a given step. DISCO, by contrast, is bounded less by the particle count than by the resolution of its force mesh: because its Poisson solve is a single global FFT over the whole PM grid (Sec. 2.2.1), rather than a spatially local tree walk,

the entire mesh – the density field, its Fourier transform, and the resulting force components – must reside in memory simultaneously for a single force evaluation, so that (unlike PKDGRAV3) its memory demand scales with the chosen mesh resolution rather than directly with the particle count. Production-resolution runs of either code therefore require a cluster.

4.1.2 Euler (ETH Zurich)

On Euler we built a separate virtual environment for each code, because DISCO and PKDGRAV3 have conflicting library dependencies that cannot coexist in a single environment: DISCO is built on JAX (Sec. 2.2.1) and therefore needs a matched GPU-software stack for its own automatic-differentiation and GPU-acceleration machinery, while PKDGRAV3 is a compiled tree code with its own domain-decomposition layer (Sec. 2.2.2) built against a separate toolchain; installing both into one environment risks exactly the kind of version conflict that a per-code environment avoids. Euler provides the memory and the GPU partitions needed to run at the full 832^3 resolution, with substantial speed-ups from the GPU-accelerated builds – speed-ups that matter specifically for DISCO, whose entire force solve is GPU-accelerated by construction (Sec. 2.2.1), rather than for PKDGRAV3, whose FMM force calculation is CPU-bound (in the used version). Depending on the configuration a single simulation can still be long-running, so jobs are distributed across multiple nodes where the code supports it.

4.1.3 CSCS (Alps / Piz Daint)

The largest runs use the CSCS Alps system, whose NVIDIA GH200 nodes provide 120 GB of on-package memory. This large memory pool is well suited to scaling DISCO up in particular: as argued in Sec. 4.1.1, DISCO’s memory demand tracks the resolution of its PM grid rather than the particle count, and the entire grid must be resident in memory for a single force evaluation, so a larger available memory directly translates into a finer mesh – and hence a smaller-scale force resolution – that can be run without running out of memory mid-simulation. The same large memory pool also shortens the wall-clock time of each run, whether generating the DISCO/CosmoGridV1 training pairs of Sec. 3.1 or, later, training the correction network of Sec. 3.4 itself, since both tasks can be run with larger batches – of simulations in the first case, of map patches in the second – without being forced to break the workload into smaller pieces purely to fit within a tighter memory budget.

4.2 Software Engineering

4.2.1 Code Structure

Executables and virtual environments are kept in the cluster’s **HOME** directory, which is persistent and backed up but subject to a strict storage quota. This split is deliberate: the code and its environment are exactly the things that must survive between sessions and be rebuildable on demand, so they belong in the small, safe, backed-up **HOME**, whereas simulation snapshots, shell maps, per-job logs, and diagnostic plots are bulk, regenerable output that is instead written to **SCRATCH**, which trades that persistence for effectively unbounded (though periodically purged) capacity – exactly the trade-off that disposable intermediate output should make. Keeping the full log of every submission proved valuable both for debugging and for later cross-run comparison, since a run’s log is often the only remaining record of exactly which configuration produced a given output. The CosmoGridV1 reference dataset (~ 110 TB) was transferred to the CSCS entry point with a dedicated Globus transfer job: at this scale, an interactive, single-stream copy would be both too slow and too fragile, since a single dropped connection partway through could force the transfer to restart from scratch, whereas a managed transfer job can run unattended and recover from such interruptions on its own.

4.2.2 Job Scheduling and Submission

4.2.3 Job Scheduling and Submission

Every Python script is paired with a Slurm batch script. The Python entry point is written to run unchanged both locally and on the cluster, while the accompanying batch script requests the resources and captures the output; keeping the two separate means the same driver used for the small local tests of Sec. 4.1.1 is also what eventually runs, unmodified, on a full production allocation, with only the surrounding batch script changing. The same job can then be launched interactively (**bash**, for fast monitoring directly in the terminal) or submitted to the queue (**sbatch**, to run detached and log to file) – interactive launches being the natural continuation of local debugging once a job needs cluster-scale resources, and queued submission being reserved for runs that are expected to actually take long enough to justify running unattended.

Each batch script follows the same two-part layout. The first part is a header of **#SBATCH** directives that declares the resource request to the scheduler, and the second is a short execution body that prepares the environment and launches the driver. The header groups naturally into three concerns:

- **Resources** – how much hardware to reserve: the number of **nodes**, the GPUs per node (**gres=gpu**), the task layout (**ntasks-per-node**), **exclusive** node access, and a wall-clock **time** limit after which the job is killed.
- **Accounting** – where the allocation is charged and queued: the **account** and the **partition**.
- **Bookkeeping** – where results and diagnostics go: a human-readable **job-name**, the working directory (**chdir**), and the **output/error** log paths, where **%j** expands to the unique job ID so that concurrent runs never overwrite one another.

The **Resources** block also fixes what the job costs against the allocation: requesting n_{nodes} nodes for a wall-clock time t is billed as

$$C = n_{\text{nodes}} \times t \quad [\text{node-hours}], \quad (20)$$

regardless of how much of that time is actually spent computing rather than sitting idle, which is exactly why the interactive/queued distinction above matters in practice: short, single-node interactive sessions are used for debugging, where the actual compute is brief and the goal is a fast turnaround rather than efficient use of Eq. (20), while **exclusive**, multi-node allocations with a generous **time** limit are reserved for production runs, where the requested resources are what the run is actually expected to use. A representative submission script is shown below.

The execution body is short, but it is doing three distinct jobs before the driver ever starts. Giving here an concrete example of the DISCO run. First, it activates the DISCO-specific environment (**miniforge/conda activate vir_env**) – the per-code environment split motivated in Sec. 4.1.1, made concrete here as the very first thing the job does. Second, it fixes the paths the driver needs: **IC_FILE** points to **CosmoML.tipsy**, the backscaled PKDGRAV3 initial condition of Sec. 3.2, handed to DISCO in the **tipsy** format, and **PARAM_FILE** points to the run’s configuration. Third, it sets the physical run parameters as ordinary shell variables rather than burying them in the driver itself, so that the same script is reused for different configurations. **RES=832** and **BOXSIZE=900.0** are exactly the CosmoGridV1 particle count and box size already fixed in Sec. 4.1.1, while **RES_PM=1664** sets DISCO’s particle-mesh grid to exactly twice that resolution per dimension ($1664 = 2 \times 832$) – the finer force mesh whose memory cost is exactly what the estimate in Sec. 4.1.1 is built around.

The task count passed to **srun** is computed, rather than hard-coded, for the same reason. The header itself only reserves **--ntasks-per-node=1**, so Slurm’s own bookkeeping treats the job as one task per node; the actual degree of parallelism – one process per GPU – is left to the execution body, which reads the number of nodes Slurm actually granted (**SLURM_NNODES**) and multiplies it by the four GPUs requested per node (**--gres=gpu:4**) to get **NTASKS=\$((SLURM_NNODES * 4))**. Passing this computed value explicitly to **srun --ntasks** is what lets **--nodes** in the header be the only thing that needs to change to scale the

same script from the two-node example shown here to a larger allocation: the task count, and hence the process-per-GPU launch, follows automatically.

Because only the header and these few shell variables change between runs, adapting a job to a new configuration – more nodes, a different resolution, a longer time limit, or a different output directory – amounts to editing a handful of lines, while the underlying Python driver invoked by `simulation_run` is reused unchanged.

4.2.4 Version Control (GitHub)

The whole project – the analysis scripts together with the modified DISCO that includes the shell builder – is kept in a dedicated Git repository. Beyond the usual protection against machine failure, this gives a reliable way to return to a known-good state when an experimental change breaks the pipeline, and keeps the many configuration variants organised.

5 Results

6 Conclusion

References

- [1] J. M. Bardeen et al. “The Statistics of Peaks of Gaussian Random Fields”. In: *The Astrophysical Journal* 304 (May 1986), p. 15. ISSN: 0004-637X. DOI: 10.1086/164143. URL: <https://ui.adsabs.harvard.edu/abs/1986ApJ...304...15B/abstract> (visited on 07/07/2026).
- [2] J. Bond, Lev Kofman, and D. Pogosyan. “How filaments of galaxies are woven into the cosmic web”. In: *Nature* 380 (Apr. 18, 1996), pp. 603–606. DOI: 10.1038/380603a0.
- [3] Michaël Defferrard et al. *DeepSphere: a graph-based spherical CNN*. Dec. 30, 2020. DOI: 10.48550/arXiv.2012.15000. arXiv: 2012.15000[cs.LG]. URL: <http://arxiv.org/abs/2012.15000> (visited on 07/02/2026).
- [4] Scott Dodelson and Fabian Schmidt. “Modern Cosmology”. In: *Modern Cosmology* (2020). ISBN: 9780128159491. DOI: 10.1016/C2017-0-01943-2. URL: <https://ui.adsabs.harvard.edu/abs/2020moco.book.....D/abstract> (visited on 07/04/2026).
- [5] Daniel J. Eisenstein and Wayne Hu. “Baryonic Features in the Matter Transfer Function”. In: *The Astrophysical Journal* 496.2 (Apr. 1998), pp. 605–614. ISSN: 0004-637X, 1538-4357. DOI: 10.1086/305424. arXiv: astro-ph/9709112. URL: <http://arxiv.org/abs/astro-ph/9709112> (visited on 07/07/2026).
- [6] `example/pkdgrav3_lightcone · main · Janis Fluri / Pkdgrav3_intro · GitLab`. GitLab. Feb. 2, 2022. URL: https://cosmo-gitlab.phys.ethz.ch/jafluri/pkdgrav3_intro/-/tree/main/example/pkdgrav3_lightcone (visited on 07/04/2026).
- [7] Janis Fluri et al. “A Full Λ CDM Analysis of KiDS-1000 Weak Lensing Maps using Deep Learning”. In: *Physical Review D* 105.8 (Apr. 20, 2022), p. 083518. ISSN: 2470-0010, 2470-0029. DOI: 10.1103/PhysRevD.105.083518. arXiv: 2201.07771[astro-ph.CO]. URL: <http://arxiv.org/abs/2201.07771> (visited on 07/02/2026).
- [8] Oliver Hahn, Florian List, and Natalia Porqueres. “DISCO-DJ I: a differentiable Einstein-Boltzmann solver for cosmology”. In: *Journal of Cosmology and Astroparticle Physics* 2024.6 (June 1, 2024), p. 063. ISSN: 1475-7516. DOI: 10.1088/1475-7516/2024/06/063. arXiv: 2311.03291[astro-ph.CO]. URL: <http://arxiv.org/abs/2311.03291> (visited on 07/02/2026).

- [9] Dragan Huterer. “Growth of Cosmic Structure”. In: *The Astronomy and Astrophysics Review* 31.1 (Dec. 2023), p. 2. ISSN: 0935-4956, 1432-0754. DOI: 10.1007/s00159-023-00147-4. arXiv: 2212.05003[astro-ph.CO]. URL: <http://arxiv.org/abs/2212.05003> (visited on 07/07/2026).
- [10] Tomasz Kacprzak et al. “CosmoGridV1: a simulated Λ CDM theory prediction for map-level cosmological inference”. In: *Journal of Cosmology and Astroparticle Physics* 2023.2 (Feb. 1, 2023), p. 050. ISSN: 1475-7516. DOI: 10.1088/1475-7516/2023/02/050. arXiv: 2209.04662[astro-ph.CO]. URL: <http://arxiv.org/abs/2209.04662> (visited on 07/02/2026).
- [11] Julien Lesgourgues and Sergio Pastor. “Massive neutrinos and cosmology”. In: *Physics Reports* 429.6 (July 2006), pp. 307–379. ISSN: 03701573. DOI: 10.1016/j.physrep.2006.04.001. arXiv: astro-ph/0603494. URL: <http://arxiv.org/abs/astro-ph/0603494> (visited on 07/07/2026).
- [12] Julien Lesgourgues and Sergio Pastor. “Neutrino mass from Cosmology”. In: *Advances in High Energy Physics* 2012 (2012), pp. 1–34. ISSN: 1687-7357, 1687-7365. DOI: 10.1155/2012/608515. arXiv: 1212.6154[hep-ph]. URL: <http://arxiv.org/abs/1212.6154> (visited on 07/07/2026).
- [13] Noam I. Libeskind et al. “Tracing the cosmic web”. In: *Monthly Notices of the Royal Astronomical Society* 473.1 (Jan. 1, 2018), pp. 1195–1217. ISSN: 0035-8711, 1365-2966. DOI: 10.1093/mnras/stx1976. arXiv: 1705.03021[astro-ph.CO]. URL: <http://arxiv.org/abs/1705.03021> (visited on 07/04/2026).
- [14] Florian List et al. “DISCO-DJ II: a differentiable particle-mesh code for cosmology”. In: *Journal of Cosmology and Astroparticle Physics* 2026.6 (June 1, 2026), p. 033. ISSN: 1475-7516. DOI: 10.1088/1475-7516/2026/06/033. arXiv: 2510.05206[astro-ph.CO]. URL: <http://arxiv.org/abs/2510.05206> (visited on 07/02/2026).
- [15] Douglas Potter, Joachim Stadel, and Romain Teyssier. *PKDGRAV3: Beyond Trillion Particle Cosmological Simulations for the Next Era of Galaxy Surveys*. arXiv.org. Sept. 27, 2016. URL: <https://arxiv.org/abs/1609.08621v1> (visited on 07/03/2026).
- [16] Uros Seljak and Matias Zaldarriaga. “A Line of Sight Approach to Cosmic Microwave Background Anisotropies”. In: *The Astrophysical Journal* 469 (Oct. 1996), p. 437. ISSN: 0004-637X, 1538-4357. DOI: 10.1086/177793. arXiv: astro-ph/9603033. URL: <http://arxiv.org/abs/astro-ph/9603033> (visited on 07/07/2026).

A Implementation Details

A.1 Job Configuration

Listing 1 SLURM job script for a DISCO run.

```
#!/bin/bash

# ===== Job settings =====

# --- Resources ---
#SBATCH --nodes=2
#SBATCH --exclusive
#SBATCH --ntasks-per-node=1
#SBATCH --gres=gpu:4
#SBATCH --time=01:00:00

# --- Accounting ---
#SBATCH --account=sk037
#SBATCH --partition=normal

# --- Bookkeeping ---
#SBATCH --job-name=run_disco
#SBATCH --output=../run_disco-%j.out
#SBATCH --error=../run_disco-%j.err

# ===== Execution body =====

# 1. Activate the code-specific virtual environment
source miniforge/bin/activate
conda activate vir_env

# 2. Paths to input files
IC_FILE=../CosmoML.tipsy
PARAM_FILE=../params.yml
# ... additional paths as needed

# 3. Simulation parameters
RES=832
RES_PM=1664
BOXSIZE=900.0
# ... additional parameters as needed

# 4. Run the simulation
NTASKS=$((SLURM_NNODES * 4))

srun --ntasks="${NTASKS}" \
    simulation_run \
    --ics-file "${IC_FILE}" \
    --param-file "${PARAM_FILE}" \
    --res "${RES}" \
    --res-pm "${RES_PM}" \
    --boxsize "${BOXSIZE}" \
    # ... additional flags
```